

AI Agent 全栈工程师训练营 课程大纲

课程定位

本课程面向希望转型为 **AI Agent 开发工程师 / 大模型应用开发工程师 / RAG & Agent 工程师** 的传统后端、全栈或平台工程师。

课程不以低代码平台操作或抽象行业概念为核心，而以“**如何构建生产级 Agent 系统**”为主线，系统训练：

- LLM API 调用、Prompt、结构化输出与模型治理；
- Function Calling、Tool Runtime、MCP 工具协议与工具安全边界；
- Agent Loop、状态机、长程任务执行、Sandbox 与失败恢复；
- Context Engineering、Memory、RAG 与知识库治理；
- Multi-Agent、Skill、Agent Eval、Benchmark 与回归测试；
- Trace、Replay、灰度发布、成本治理、生产部署与 Runbook；
- 企业级深度研究平台、软件工厂等高阶 Agent 应用架构。

课程最终目标不是让学员“会调用模型”或“能搭 Demo”，而是让学员具备独立构建、评测、部署和持续迭代 Agent 系统的工程能力。

Agent 开发工程师必备核心能力

AI Agent 工程师岗位汇总

懂后端系统稳定性，能把模型能力封装成稳定服务。

懂工具调用和权限边界，能避免 Agent 误用工具或越权执行。

能做长程任务 Agent，而不是只调用一次模型。

能优化上下文、记忆和任务状态管理。

能建立评测体系，用 benchmark 和数据证明 Agent 变好。

能把模型能力产品化，而不是停留在 Demo。

课程核心价值

岗位导向：课程面向 AI Agent 开发工程师、大模型应用开发工程师、RAG / Agent 工程师岗位中最核心的硬技能要求。

工程导向：重点训练工程视角下的 Agent 开发能力，包括工具注册、参数校验、权限控制、Trace、失败恢复、测试评估。

作品集导向：课程围绕 Agent Tool Runtime、Codebase Agent、Agent Eval Platform 三个作品集项目展开，让学员有真实可展示的工程成果。

可迭代导向：强调 Eval 是 Agent 的核心护城河，要求学员能够构造 benchmark、记录成功率、分析失败样本，并基于数据迭代 Agent。

真实使用导向：要求学员用自己的 Agent 辅助真实开发、代码理解、测试生成、失败修复和文档输出，让 Agent 成为日常开发工具。

学习收益

核心能力模型认知建立：能够了解目前企业中 AI Agent 开发岗位所需的技能需求，依据能力模型进行特定知识点的提升，构建自身学习路径。

LLM 工程基础：掌握 LLM API、Prompt、Function Calling、Structured Output、Streaming、Token / Context Window、Cost / Latency 等基础能力。

Agent Tool Runtime：能够设计工具注册中心、JSON Schema 参数校验、权限控制、调用审计、超时重试、Streaming 返回和 Trace 可视化。

长程任务 Agent：能够实现 observe -> think / plan -> act -> observe -> finalize 的 Agent Loop，掌握从 ReACT 到 Loop 的主流 Agent 进化路径。

上下文与记忆管理：掌握 Context Manager，完成长文件摘要、历史工具结果压缩、任务状态持久化、长期记忆和项目级记忆。

多 Agent 与评测体系：能够设计 Subagent 协作、结果聚合、冲突处理、Agent benchmark、自动化评测、Trace Analysis 和回归测试。

期望用户学完以后可以拥有一个统一、完整、可展示的 **Agent Engineering Platform**

平台至少包括：

- LLM Gateway;
- Prompt 与 Structured Output;
- Tool Runtime;
- MCP Tool 接入;
- Codebase Agent;
- Agent Loop;

- State Machine;
- Sandbox;
- Context Manager;
- Project Memory;
- Codebase RAG;
- Multi-Agent;
- Skill System;
- Agent Eval Platform;
- Trace;
- Replay;
- 成本统计;
- 灰度发布;
- Docker Compose 部署;
- 上线 Runbook。

最终可围绕该平台沉淀三类求职作品集成果：

1. Agent Tool Runtime：体现工具治理、权限、安全与后端工程能力；
2. Codebase Agent：体现长程任务、代码理解、测试修复、RAG 与 Context Engineering 能力；
3. Agent Eval & Production Platform：体现评测、可观测性、版本管理、成本治理与生产化能力
4. 深度研究以及软件工厂Agent项目

第一章：LLM API、Prompt 与结构化输出

本章目标：建立大模型应用开发的基础认知，完成从传统 HTTP API 调用到 LLM API 调用的迁移。本章只解决“如何稳定调用模型”的问题。

1. AI Agent 开发工程师能力模型

- 岗位能力拆解
 - 后端工程基础
 - Python 工程能力
 - LLM API 与 Prompt
 - RAG 与知识库
 - Agent Loop 与 Tool Use

- Context / Memory
- Agent Eval
- 工程化、部署与可观测性

● 转型路径

传统后端工程师的已有优势：

- API 服务开发
- 数据库与缓存
- 异步任务
- 权限认证
- 日志监控
- 限流、重试、熔断
- Docker 与部署

需要重点补齐的 Agent 能力：

- 模型调用
- Prompt 与结构化输出
- 工具协议
- 上下文管理
- Agent 状态机
- Eval 与 Agent 质量治理

2. LLM API 与模型调用基础

- Chat Completions / Responses API：理解主流模型 API 的请求结构、消息格式、模型参数和返回结果
- 模型参数：temperature、top_p、max_tokens
- Token 与 Context Window
- Structured Output：使用 JSON Schema、Pydantic 或模型原生结构化输出能力，让模型返回可解析、可校验的数据
- 错误类型与异常处理：处理模型调用失败、超时、限流、输出格式错误和重试策略
- 多模型调用基础

3. Streaming 流式输出

- 为什么 LLM 应用需要 Streaming：模型推理延迟高，串行等待降低用户体验
- Server-Sent Events：基于 HTTP 长连接的流式传输协议，实现打字机效果
- FastAPI 流式接口：使用 `StreamingResponse` 保持低延迟转发
- 前端 / CLI 消费流式输出：EventSource / ReadableStream 逐块消费
- 中断与取消：用户可随时取消生成，避免浪费 Token
- 流式输出异常处理：网络抖动或限流导致中断时的捕获与重试
- 长文本输出的状态保存：超出 Context Window 或会话断开时持久化断点

4. Prompt Engineering

- System Prompt 的职责：设定全局行为基调，相当于模型的"岗位说明书"
- 角色定义与行为约束：明确告诉模型扮演什么角色（代码审查助手 / 分析师等）
- 输出风格控制：控制详细程度、语气、结构，确保风格符合场景
- Few-shot 示例：给 2-3 个"输入→输出"范例，对格式敏感任务更有效
- 任务拆解：将复杂任务拆成多步执行，降低幻觉率和逻辑跳跃
- Prompt 模板：将可变参数嵌入固定模板，使 Prompt 可复用、可版本管理
- Prompt 注入风险：用户输入篡改指令的风险及防护措施
- Prompt 版本管理的基本思想

5. Structured Output 结构化输出

- 为什么 Agent 系统不能依赖自然语言解析
- JSON Schema：定义输出结构的标准规范（字段名、类型、是否必填等）
- Pydantic Model：定义输出结构 + Schema 校验 + 反序列化为 Python 对象
- 模型原生结构化输出：API 层直接传入 Schema，模型保证输出合规
- Schema 校验：模型返回后校验 JSON 是否符合预期结构
- 输出纠错与重试：校验失败时把错误反馈给模型自行修正
- 结构化输出失败处理：多次重试失败后降级或回退到缓存结果
- 输出格式与业务协议设计

6. LLM Gateway

打造一个 Agent Engineering Platform，在平台中完成：

- FastAPI LLM 服务：统一 HTTP 入口，各组件无需各自维护 API 密钥
- 模型调用接口：封装 OpenAI Compatible 协议，统一多厂商格式差异
- Streaming 输出：Gateway 层代理流式响应，逐块转发
- Structured Output：集成 Schema 传入机制，屏蔽底层实现差异
- Pydantic 数据校验：请求入口和响应出口各做一次校验拦截
- Prompt 模板：Gateway 层管理模板库，上层只需传入变量
- 错误处理与重试：统一处理限流、超时，自动 fallback 到备用模型
- Token / Cost / Latency 日志：每次调用留痕，为成本治理提供数据

7. 项目演进路线 阶段一：LLM 统一模型调用服务

- **项目背景：**从传统 HTTP API 调用到 LLM API 调用的迁移，掌握了 Prompt Engineering、Structured Output、Streaming 等基础能力后，需要构建一个统一的模型调用服务作为 Agent Engineering Platform 的入口层。
- **核心要点：**
 - 构建 OpenAI Compatible API 接口
 - 多模型调用与模型路由
 - Streaming 流式输出
 - Structured Output 结构化输出
 - Prompt 模板管理
 - Token、Cost、Latency 记录
 - 基础错误处理、重试与限流

第二章：Function Calling、Tool Runtime 与 MCP

本章目标：理解 Agent Tool Use 原理，掌握一套包含工具注册、参数校验、权限控制、审计、超时、失败恢复和可观测性的运行时基础设施。

1. Function Calling 与 Tool Use

- Function Calling 工作机制
- 工具定义：名称、描述、参数 Schema、返回 Schema、错误 Schema、权限等级和风险等级
- Input Schema、Output Schema、Error Schema

- 工具选择：模型根据用户请求自动选择工具，并生成符合 Schema 的参数
- 工具治理：参数校验、权限控制、调用审计、超时、重试、失败恢复和 Trace 记录

2. Tool Runtime 设计

• Tool Registry

- 工具注册：支持注册查询天气、查数据库、调用内部 HTTP API、文件处理等工具。
- 工具发现：让 Agent 能根据任务找到可用工具
- 工具元数据：记录工具说明、参数、返回值、权限、风险等级等信息
- 工具分类：按业务类型、读写属性、风险级别管理工具
- 工具版本：支持工具升级、兼容旧版本和灰度切换
- 工具依赖：声明工具运行所需的服务、密钥、数据库或外部资源
- 工具启停管理：支持临时下线高风险或异常工具

• Tool Schema

- 每个工具至少应包含：工具名称、工具说明、输入参数、输出结构、错误结构、权限级别、风险级别、超时配置、重试策略、审计字段

3. 工具治理与安全边界

- 参数校验；
- 权限隔离：区分只读工具、写操作工具、高风险工具和需要确认的工具
- RBAC / 用户身份透传
- 只读工具与写操作工具
- 高风险工具
- 工具白名单
- SQL / Shell 风险控制
- 人工确认机制
- 工具调用队列：管理多个工具调用的顺序、依赖和结果
- 工具调用审计
- 敏感数据脱敏
- 超时与失败恢复：工具失败后进行重试、参数修正、降级或人工兜底

4. MCP: Model Context Protocol

• MCP 基础

MCP 是连接模型与外部工具 / 资源的标准协议。Agent 通过 MCP Client 发现并调用 MCP Server 暴露的工具、资源和 Prompt，例如文件系统、数据库、GitHub 或内部 API。Function Calling 负责“模型如何选择工具”，MCP 负责“工具如何标准化接入”。相比传统 HTTP API，MCP 更适合 Agent 的工具发现、资源读取和多工具组合场景。

• MCP 工具接入

- 接入文件系统 MCP
- 接入数据库 MCP
- 接入 GitHub MCP
- 接入内部 HTTP 服务
- MCP 工具权限控制
- MCP 工具审计
- MCP 连接管理

5. Agent Tool Runtime

打造一个 Agent Engineering Platform，在平台中完成：

- Tool Registry：统一注册和管理所有工具
- Function Calling：让模型根据用户任务选择合适工具
- JSON Schema 参数校验：确保模型生成的工具参数可解析、可执行
- Pydantic 工具参数模型：用代码模型承接工具输入输出结构
- 工具权限控制：限制 Agent 在不同身份和场景下的可调用范围
- 风险等级：按只读、写操作、高风险操作划分工具安全级别
- 工具调用审计：记录每次工具调用的输入、输出、状态和 Trace
- 超时、重试、失败返回：保证工具异常时系统不会卡死
- MCP Tool 接入：将外部工具和数据源通过 MCP 统一接入
- Tool Call Trace：追踪工具从选择、参数生成、执行到返回的完整链路
- CLI 工具调用入口：提供命令行方式测试和调试工具调用能力

6. 项目演进路线 阶段二：Agent 工具调用基础设施

- **项目背景：** 学员掌握了 Function Calling 机制、工具注册与 Schema 定义、MCP 协议等核心概念后，需要构建一套生产级的工具调用运行时基础设施。
- **核心要点：**
 - ToolRegistry 工具注册中心
 - ToolSchema 工具元数据定义
 - JSON Schema / Pydantic 参数校验
 - Function Calling 集成
 - 权限校验与风险分级
 - 工具调用审计
 - 超时、重试、降级机制
 - MCP Client 接入
 - 工具调用 Trace 记录

第三章：Agent Loop、State Machine 与 Codebase Agent

本章目标： 从“单轮模型调用 + 单次工具执行”升级为可连续思考、执行、观察、纠错和结束的长程任务 Agent。本章聚焦任务执行过程、状态机、执行环境与失败恢复能力。

1. 从 Tool Agent 到 Agent Loop

- 单轮 Tool Calling 的局限
- 为什么复杂任务需要多步执行
- ReAct 模式
- Plan-and-Execute
- Observe → Think / Plan → Act → Observe → Finalize
- 最大循环次数
- 循环终止条件
- 死循环识别
- 任务中断与恢复

2. Planning 与任务拆解

- 一次性计划：适合目标明确、步骤稳定的任务
- 动态计划：适合执行过程中信息不断变化的长程任务

- Plan Revision
- 子目标拆解
- 步骤依赖
- 并非所有任务都需要 Planner
- 何时适合固定 Workflow
- 何时适合 Agent Loop
- 计划与实际执行偏差处理：根据工具结果、测试失败或上下文变化调整下一步动作

3. State Machine 与 Checkpoint

- **任务状态**：created、planning、running、tool_failed、waiting_tool、waiting_approval、completed、failed、cancelled
- **步骤状态**：当前步骤、工具名称、输入参数、工具输出、失败原因、重试记录、下一步决策、任务最终结果
- **Checkpoint**：为什么 Agent 需要 Checkpoint、任务中断恢复、状态持久化、失败任务复现、人工接管后继续执行

4. Sandbox 与执行边界

Agent 一旦具备文件读写、Shell 执行、网络访问等能力，就必须被限制在安全边界内。本节重点解决“Agent 能做什么、不能做什么、出了问题如何追踪”的问题，让 Codebase Agent 可以安全地执行真实开发任务。

- 文件系统边界：区分可读目录、可写目录和禁止访问目录
- Shell 命令控制：通过命令白名单、超时限制和高风险命令拦截，避免误删文件或执行危险操作
- 网络访问控制：限制 Agent 是否可以访问外网、内网服务或第三方 API
- 本地 Sandbox：适合教学、调试和轻量任务执行
- Docker Sandbox：适合隔离执行环境，避免 Agent 影响宿主机
- 本地与容器边界差异：理解文件、网络、权限和环境变量在两种 Sandbox 中的不同
- 工具审计与风险回放：记录每次命令、文件操作和执行结果，方便复盘与定位问题

5. Agent Harness

Agent Harness 是把模型、工具、状态、上下文和执行环境组织在一起的运行框架。它让 Agent 不再只是“一次模型调用 + 一次工具执行”，而是一个可配置、可追踪、可恢复的任务执行系统。

- Harness 的定义
- 模型、工具、状态、上下文和执行环境的统一组织
- 从单次 Tool Use 到 Agent Runtime
- Agent 生命周期管理
- Agent 配置
- 工具集装箱
- 上下文输入
- Checkpoint+Trace
- Human-in-the-loop：在高风险动作前请求用户确认，例如修改文件、删除数据、发起外部请求
- Harness 化的工程意义

6. LangGraph 与 Agent 编排

- LangGraph：对象（图（节点、边、状态））
- StateGraph
- 条件分支与循环
- Checkpoint
- Interrupt
- Tool Node
- 调试与可视化
- LangGraph 与传统 Workflow 的关系
- LangGraph、OpenAI Agents SDK、AgentScope 的定位对比
- 通过 Dify 理解流程节点和条件分支、状态管理、人机协同、自动化业务流程，LangGraph 状态图

7. Codebase Agent

学会拆解业务流程

- 从业务 SOP 拆 Agent Loop
- 从岗位任务拆 Tool / Memory / Workflow
- 从用户旅程设计人机协同节点

在此基础上构建第一版可执行代码仓库任务的 Agent。

支持工具

- list_files、search_code、read_file、read_directory、write_file、apply_patch、run_test、run_command、git_diff、git_status

典型任务

任务一：代码理解

“帮我找出该项目的登录逻辑，并生成 Markdown 说明文档。”

Agent 能够：

- 搜索认证相关代码
- 阅读路由、中间件、Service、数据库逻辑
- 梳理调用链
- 输出模块说明
- 给出相关文件路径

任务二：测试生成与修复

“给指定函数补充测试，并根据测试失败日志进行修复。”

Agent 能够：

- 定位函数
- 分析已有测试、编写测试、执行测试
- 读取失败日志
- 修正测试或代码
- 输出变更说明

8. 项目演进路线 阶段三：Codebase Agent

项目背景： 从"单轮模型调用+单次工具执行"升级到可连续思考、执行、观察、纠错的长程任务 Agent，需要构建一个能在代码仓库中完成复杂任务的 Agent 系统。

核心要点：

- Agent Loop 循环执行框架
- Planning 任务规划
- 搜索、读取、修改文件能力
- 执行测试与读取失败日志
- 自动修复错误

- Human-in-the-loop 人工确认机制
- Sandbox 安全执行环境
- 任务状态管理与 Checkpoint

第四章：Context Engineering、Memory 与 Codebase RAG

本章目标：理解 Agent 的能力上限不只由模型决定，也取决于它在每轮任务中“看到了什么信息、遗漏了什么信息、如何压缩信息、如何检索知识”。本章主要解决上下文、知识和记忆问题。

1. Context Engineering

- Prompt Engineering 与 Context Engineering 的区别
- 上下文长度：理解模型上下文窗口限制，以及长上下文带来的成本和延迟问题
- Context Packing：判断哪些信息应该进入上下文，哪些信息应该留在外部状态
- Context Prioritization
- Context Budget
- Context Compression：长文件截断与摘要、工具结果压缩、历史对话压缩、无关信息过滤
- 上下文污染
- Token 预算
- 成本与延迟：通过缓存、并发、限流、重试和模型路由控制成本与响应时间
- Context Debug Report
- Retrieval Context：从文档、代码、历史记录中找相关信息
- KV Cache：不用深入训练层，但要理解它如何影响长上下文推理成本和延迟
- LiteLLM：通过 LiteLLM 演示模型路由、fallback、调用日志和成本追踪

2. Working Memory 与任务上下文

- Working Memory：保存当前任务相关信息，例如目标、已完成步骤、工具结果和未解决问题
- 当前风险与用户约束
- 临时事实记录
- Scratchpad：保存 Agent 中间推理、执行状态和下一步计划
- 工作记忆的生命周期
- 工作记忆与 Agent State 的边界

3. Memory 体系

- 短期记忆：当前会话与当前任务状态
- 长期记忆：用户偏好、历史任务、常用工具、个性化配置、记忆写入、记忆检索、记忆更新、记忆删除、记忆可信度、项目约定和常用策略
- 项目级记忆：
 - 项目目录结构：技术栈、模块边界、构建命令、测试命令、部署方式、编码规范、常见故障、历史修复经验、项目约定、和开发约定
- 任务摘要与状态恢复：让 Agent 能在上下文被截断后继续执行任务
- Memory 策略对比：比较不同记忆策略对任务成功率、成本和延迟的影响

4. 现代 RAG 系统

文档解析：PDF、Markdown、HTML、表格与结构化文档、文档清洗、元数据提取。

- **Chunking:**
 - 固定长度分块、按段落分块、按标题分块、语义分块、代码 AST 分块、Chunk Overlap、Chunk Metadata
- **检索**
 - Embedding：向量化模型选择、维度、召回质量
 - Vector Database：FAISS、Milvus、Chroma 至少熟悉一种
 - BM25
 - Elasticsearch
 - Hybrid Search：向量检索 + BM25 / Elasticsearch
 - Metadata Filter
 - Query Rewrite：把用户问题改写成更适合检索的问题（查询重写、分解、澄清）
 - Query Decomposition
 - 查询澄清
- **检索后处理**
 - Rerank: Cross Encoder / LLM Rerank
 - 上下文压缩
 - 证据筛选

- Citation：回答必须引用来源
- 回答与证据一致性校验
- 检索后处理：重排、压缩、校正
- **RAG 评估基础**
 - 召回率、命中率、答案正确率、幻觉率

5. LLM Wiki 与知识资产治理

- LLM Wiki 的定义
- LLM Wiki 与传统知识库的差异
- LLM Wiki 与 RAG 的关系
- 知识分类：按制度、产品、技术、调研资料、项目文档等方式组织知识
- 文档版本：记录知识更新历史，避免 Agent 使用过期内容
- 来源追踪：保留文档来源、作者、时间和引用路径
- 知识维护：定期清理重复、过期、低质量或冲突内容
- 知识权限：控制不同用户、部门或 Agent 可访问的知识范围
- 知识质量：评估知识完整性、可信度、时效性和可检索性
- 企业制度、产品文档、技术文档、调研资料的组织方式

6. Codebase RAG & Memory

为 Codebase Agent 增加代码库检索、上下文管理和项目记忆。

- **实现能力**
 - 代码库索引+文档索引
 - 代码 AST / 文件级 Chunk
 - 向量检索+BM25 检索
 - Hybrid Search
 - Rerank
 - 文件引用+行号引用
 - 长文件摘要
 - 工具结果压缩
 - 项目级记忆

- Context Debug Report
- 可恢复任务摘要
- **典型任务**
 - “该项目的鉴权链路是怎样的？”
 - “如果新增一个支付渠道，需要改哪些模块？”
 - “找出与订单状态流转相关的全部代码和文档。”
 - “该接口可能存在哪些性能瓶颈？”
 - “解释这个异常的根因，并给出相关代码证据。”

7. 项目演进路线 阶段四：Codebase RAG & Memory

项目背景： 理解了 Agent 的能力上限取决于它能"看到什么信息、遗忘什么信息"后，需要为 Codebase Agent 增加上下文管理、记忆和检索增强能力。

核心要点：

- 代码库索引与文档解析分块
- 向量检索、关键词检索、混合检索
- Rerank 重排序
- 项目级记忆（目录结构、技术栈、构建命令等）
- 长文件压缩与工具结果压缩
- 任务摘要与恢复
- 引用来源追溯
- Context Debug Report

第五章：Multi-Agent、Skill 与 Agent Eval

本章目标： 理解单 Agent 与多 Agent 的边界，掌握复杂任务拆解、子 Agent 协作、结果聚合和冲突处理；同时建立可量化、可回归、可持续优化的 Agent Eval 体系。

1. Multi-Agent 与 Subagent

- 为什么不是所有任务都需要 Multi-Agent
- 单 Agent 的适用场景：目标明确、步骤较短、上下文集中、工具调用链路简单的任务
- Multi-Agent 的适用场景：代码审查、研究分析、安全扫描、性能诊断、复杂报告生成等多视角任务

- Subagent: 主 Agent 将复杂任务拆成子任务, 并为每个子 Agent 分配目标、上下文和工具
- Supervisor Pattern: 一个主控 Agent 负责任务拆解、调度、结果收集和最终决策
- Planner / Executor Pattern: Planner 负责制定计划, Executor 负责具体执行, 适合长程任务
- Reviewer Pattern: 引入审查 Agent 对结果进行复核, 降低幻觉、遗漏和错误结论
- 并行与串行执行: 独立子任务可并行执行, 有依赖关系的任务需要串行执行
- 隔离机制: 不同子 Agent 之间需要上下文隔离、工具隔离和权限隔离, 避免互相污染或越权
- 成本与超时控制: 对子 Agent 设置最大执行时间、最大调用次数和 Token 预算
- 冲突处理: 当多个子 Agent 结论不一致时, 由主 Agent 基于证据、优先级或人工确认进行仲裁

2. Subagent 任务委托

- **Task Delegation**

子任务应明确:

- 任务目标: 这个子 Agent 要解决什么问题
- 输入上下文: 它能看到哪些代码、文档、用户需求或历史结果
- 可调用工具: 限制它能使用搜索、读文件、测试、扫描等哪些工具
- 权限边界: 明确是否允许写文件、执行命令、访问外部服务
- 预期输出: 要求输出结论、证据、风险等级或建议
- 输出格式: 统一使用 JSON、Markdown 或结构化字段, 方便主 Agent 汇总
- 超时限制: 避免子任务无限执行或消耗过多成本
- 评价标准: 定义什么算完成、什么算高质量结果
- 写操作限制: 默认优先只读, 涉及修改时必须单独授权

- **Result Aggregation**

- 子任务结果合并: 将不同 Agent 的发现统一整理成完整结论
- 证据合并与引用保留: 保留代码位置、文档来源、测试结果等依据
- 去重与排序: 合并重复发现, 并按严重程度、影响范围或优先级排序
- 结论汇总: 将分散信息转化为用户可理解、可执行的最终建议
- 冲突识别与仲裁: 发现子 Agent 结论不一致时, 由主 Agent 复核证据后裁决
- 主 Agent 复核: 最终输出前检查是否遗漏任务目标、是否存在无证据结论

3. Skill 系统

- Skill 定义：将某类任务的执行方法、输入输出规范、工具依赖、Prompt 模板和质量标准封装为可复用能力包
- Skill 生命周期: Skill 注册、Skill 检索、Skill 匹配、Skill 执行、Skill 评估、Skill 版本管理、Skill 下线、Skill 复用
- Skill 注册：定义 Skill 名称、适用场景、输入参数、输出格式、依赖工具、权限边界和评价标准
- Skill 检索：根据用户任务、上下文、业务场景和历史经验，自动匹配最合适的 Skill
- Skill 编排：将多个 Skill 组合成复杂任务流程，支持顺序执行、并行执行、条件分支和人工确认
- Skill 复用：在不同 Agent、不同业务场景和不同项目中复用成熟 Skill，减少重复 Prompt 和重复流程设计

4. Agent Eval: Agent 的核心护城河

- 为什么需要 Eval
 - Agent 能运行不代表 Agent 可靠
 - Demo 成功不代表真实任务成功
 - 修改 Prompt 可能导致旧任务退化
 - 更换模型可能导致工具调用行为变化
 - RAG 策略变化可能降低引用准确率
 - 需要用数据证明 Agent 是否变好
- Golden Dataset
 - 目标：让你的 Agent 不只是能跑，而是能被评测、能迭代、能证明变好
 - Golden Dataset：构造标准任务集，覆盖代码理解、代码修改、文档检索和长程任务
- 核心指标
 - Task Success Rate：任务是否完成
 - Tool Call Accuracy：工具选得对不对，参数对不对
 - Tool Parameter Accuracy
 - Answer Correctness：答案是否事实正确
 - Citation Accuracy：引用是否支持结论
 - Regression Test：新版本 Agent 有没有退化
 - Retrieval Recall
 - Hallucination Rate
 - Human Handoff Rate

- Human Eval: 人工标注标准和评价规范
- Latency
- Token Cost
- Tool Call Count
- Failure Recovery Rate
- **Eval 方法**
 - Rule-based Eval
 - Schema-based Eval
 - Snapshot Test
 - Golden Answer
 - LLM-as-Judge: 什么时候能用, 什么时候不可靠
 - Human Eval
 - Pairwise Comparison
 - Regression Test与A/B Test
 - Trace Analysis: 失败发生在哪一步

5. Trace Analysis 与失败诊断

- Prompt 问题: 指令不清晰、约束冲突、输出格式不稳定
- 上下文问题: 关键信息缺失、无关信息过多、上下文污染
- 检索问题: 召回不到关键文档、排序错误、引用来源不可靠
- 工具选择错误: 模型选错工具或在错误时机调用工具
- 参数错误: 字段缺失、类型错误、值不合法
- 权限拦截: Agent 尝试执行超出权限范围的操作
- 工具执行失败: 超时、接口异常、外部服务不可用
- Agent Loop 死循环: 重复计划、重复调用工具、无法进入终止状态
- 子 Agent 委托错误: 任务目标不清、上下文不足或权限配置错误
- 模型能力不足: 推理、代码理解或长上下文处理能力不足
- 最终答案幻觉: 结论没有证据支撑
- 引用不准确: 引用内容无法支持回答
- 评测误判: Eval 规则或 Judge 模型判断错误

6. Multi-Agent & Agent Eval Platform

• Multi-Agent 能力

- 构建以下角色：
 - Supervisor Agent：负责任务拆解、调度和最终决策
 - Architecture Agent：分析目录结构、模块边界、依赖方向和架构问题
 - Test Agent：分析测试覆盖率、测试质量和缺失测试
 - Security Agent：检查鉴权、注入、敏感信息和权限边界
 - Performance Agent：发现慢查询、循环、缓存和性能隐患
 - Documentation Agent：整理文档、注释和知识库内容
 - Report Agent：合并结果、处理冲突并生成最终报告

• 典型任务

- “请分析该项目的可维护性，并给出可执行的重构建议。”
 - 主 Agent 将任务拆解为：
 - Architecture Agent：目录结构、模块边界、依赖方向
 - Test Agent：测试覆盖率、测试质量、缺失测试
 - Performance Agent：慢查询、循环、缓存、性能隐患
 - Security Agent：鉴权、注入、敏感信息、权限边界
 - Report Agent：合并结果、处理冲突、生成最终报告

• Eval 平台能力

- Benchmark 任务集管理
- Agent 批量运行
- Prompt / Model / RAG 策略对比
- Trace 记录
- LLM-as-Judge与人工评测
- 成功率统计、成本统计、耗时统计、工具调用统计
- 失败样本聚类
- 改进前后效果对比
- 回归测试报告

7. 项目演进路线 阶段五：Multi-Agent & Eval

项目背景：掌握了单 Agent 构建能力后，需要将其升级为可协作、可评测、可迭代的多 Agent 平台，建立 Agent 质量保障体系。

核心要点：

- Supervisor Agent 监督协作
- Architecture / Test / Security / Performance / Report Agent 多角色分工
- Skill 注册与检索机制
- Benchmark 管理与自动化评测
- LLM-as-Judge 评估
- 人工评测流程
- Trace Analysis 分析
- 回归测试与 Agent 版本对比

第六章：工程化、生产部署与可观测性

本章目标：让学员从“Agent 项目能运行”升级为“Agent 系统可部署、可监控、可灰度、可回滚、可治理”。

1. 模型服务与推理部署

- **云 API、私有化与混合部署：**模型部署需根据业务场景选择云 API、私有化、专有云或混合调用方案，在模型能力、数据敏感性、成本、延迟、可运维性与合规要求之间取得平衡，保障 Agent 系统稳定、安全、可控地运行。
- **推理服务**
 - vLLM：高吞吐推理服务、PagedAttention、连续批处理、OpenAI 兼容接口、并发请求处理、显存利用率优化。
 - PagedAttention；
 - Continuous Batching；
 - OpenAI Compatible API；
 - TGI：Text Generation Inference 的部署方式、流式输出、批处理、模型加载、推理参数配置和监控指标。
 - Triton：多模型推理服务、GPU 资源管理、模型并发、动态批处理、推理性能监控。
 - 多模型推理；
 - GPU 资源利用；
 - 并发控制；

- 推理服务指标。

2. 性能与成本优化

- 缓存体系：**Prompt Cache、Semantic Cache、Embedding Cache、RAG Cache、Tool Result Cache、缓存命中率、缓存失效策略
- 批处理与并发：**Embedding Batch、批量 Rerank、Eval Batch、批量摘要、并发队列、任务优先级、排队等待、超时处理
- 模型路由与降级：**强模型与弱模型分层、模型路由、Fallback、Token Budget、小模型优先、缓存优先、人工兜底、限流、熔断、降级策略

3. Agent 可观测性

- 日志：**
 - 模型调用日志：记录模型名称、模型版本、Prompt 版本、输入 Token、输出 Token、Cost、Latency、请求状态和错误原因
 - 工具调用日志：记录工具名称、版本、参数、权限校验结果、执行耗时、返回结果、失败原因、重试次数和审计信息
- Agent Trace：**记录用户请求、任务规划、Context Build、检索结果、工具选择、工具执行、模型调用、子 Agent 调用、人工确认、策略拦截和最终输出
- Replay 机制：**保存失败任务、保存输入、保存上下文、保存工具结果、保存 Agent 状态、复现问题、验证修复、支撑回归测试

4. 监控与告警

- 业务指标：**任务成功率、自动化完成率、人工接管率、用户满意度、工单处理时长、平均任务处理时长、自动化完成率
- 模型指标：**Token 消耗、平均延迟、成本趋势、调用失败率、模型 fallback 率、P95 / P99 延迟、Fallback 率
- Agent 指标：**工具调用准确率、工具失败率、检索命中率、引用准确率、Loop 中断次数数、失败恢复成功率、子 Agent 成功率
- 安全指标：**高风险工具调用次数、越权工具调用拦截率、敏感字段访问次数、危险 SQL 拦截率、人工确认触发率
- 异常告警：**对模型失败率升高、工具失败率升高、成本异常、延迟异常、权限异常、SQL 拦截异常和人工接管率异常进行告警

5. 版本管理、灰度与回滚

- 版本对象

- 模型版本：记录不同模型、不同供应商、不同参数配置的效果差异
- Prompt 版本：记录提示词变更、适用场景、评测结果和回滚点
- 工具版本：记录工具 Schema、参数、权限、返回结构和业务逻辑变更
- Skill Version
- RAG 策略版本：记录切块策略、Embedding 模型、检索参数、Rerank 策略和知识库版本
- Knowledge Base Version
- Version：记录Workflow 版本的流程节点、条件分支、人审策略和失败恢复策略与Agent Version

- 灰度发布

- 灰度发布：按用户灰度、按部门灰度、按租户灰度、按任务类型灰度、按风险等级灰度、A/B Test、指标观察
- 通过 Feature Flag 控制 Prompt、模型、RAG 策略、Agent Planner 和工具版本

- 回滚策略

- 回滚策略：支持 Prompt 回滚、模型回滚、工具版本回滚、知识库版本回滚、Workflow 回滚和服务镜像回滚

6. MCP Server 与私有化部署

本节统一处理 MCP 服务在生产环境中的问题包括：MCP Server 的部署、鉴权、网络隔离、工具白名单、日志与审计以及内网工具接入、密钥管理、多租户隔离、API Key 不入库、配置中心、敏感字段脱敏、数据权限过滤。

7. Production Ready Agent Engineering Platform

完成平台生产化方案：

- 部署形态

- 本地开发环境：Docker Compose 启动 API、数据库、Redis、向量库和模型网关
- 企业内网环境：模型服务、业务 API、知识库、日志系统和权限系统部署在企业内网
- 混合部署：敏感数据和工具在内网，通用模型能力通过云 API 或专有云调用

- 安全与合规：API Key 不入库，密钥通过环境变量、密钥管理服务或配置中心管理。Trace 与日志需要脱敏，避免记录完整身份证、手机号、地址、合同金额等敏感字段。知识库、向量库和业务数据需要做租户隔离、部门隔离和权限过滤
- 平台化封装
 - SDK：将 Model Gateway、Context Build、Tool Call、Agent Run、Eval Runner 封装为内部 SDK，方便业务系统接入
 - 服务化：将 Agent 能力拆成可复用服务，例如模型服务、上下文服务、工具服务、工作流服务、评测服务和日志服务
 - 平台治理：统一鉴权、限流、审计、版本管理、成本分摊、质量门禁和上线审批
- 涉及工具和服务
 - FastAPI 服务；
 - Redis、PostgreSQL、Vector DB
 - MCP Server
 - Trace 服务、日志服务
 - Metrics 与监控、评测服务、多环境配置
 - Prompt / Tool / RAG / Model 版本管理
 - 灰度发布、Replay
 - 成本报表、上线 Runbook

8. 项目演进路线 阶段六：Production Ready

项目背景： 从"Agent 项目能运行"升级到"Agent 系统可部署、可监控、可灰度、可回滚、可审计"，需要让整个平台满足企业部署与运营要求。

核心要点：

- Docker Compose 部署方案
- Model Gateway 模型网关
- Redis、PostgreSQL、VectorDB 基础设施
- Trace、Metrics、Monitoring 可观测性
- Replay 回放调试
- Prompt / Model / RAG / Tool 版本管理
- Feature Flag 功能开关
- 灰度发布与回滚

- 成本治理
- 上线 Runbook

第七章：企业级综合项目与产品化落地

本章目标：通过企业级 Agent 平台案例理解成熟 Agent 系统的架构设计，并将前六章的工程能力转化为可展示、可交付、可面试表达的业务项目成果。

第一部分：DeerFlow 企业级 Agent 平台架构解析

目标：

通过九条核心架构线深度剖析 DeerFlow 源码，帮助开发者全面掌握企业级智能体平台的运行机制与设计精髓。学员将深入理解 DeerFlow 如何平衡安全性、性能与可维护性 —— 如沙箱隔离机制如何保障系统安全、中间件管道如何实现灵活的上下文控制、子代理系统如何支持复杂任务分解。更重要的是，课程将培养二次开发能力，让学员能够根据实际业务诉求定制专属 Agent、扩展工具生态、优化持久化策略，真正将 DeerFlow 转化为贴合自身场景的生产级智能体平台。

1. 请求入口 — Gateway 如何把用户输入托管成可观察，可控制的 run
2. 主智能体工厂 — 运行时配置如何变为可运行的智能体图
3. 工具组装 — 工具注册了，不代表这次 agent 就能用
4. 中间件管道 I — DeerFlow 如何在模型调用前准备上下文
5. 中间件管道 II — 如何裁决、门控和清理模型输出
6. 沙箱系统 — 工具执行的位置，以及为什么本地和容器沙箱不是同一边界
7. 子代理系统 — 如何将复杂的子任务委托给受限的完整代理
8. 技能系统 — 经验如何变成可安装、可审查、可重用的代理能力
9. 持久化、存储和检查点 — 什么可以恢复、什么可以查询、什么可以审计

第二部分：高阶案例一：企业级深度研究平台

目标：

基于 DeerFlow 搭建企业级深度研究平台，让 Agent 自动完成资料搜集、交叉验证、结构化分析和报告生成。通过 Skill 沉淀竞品分析、行业调研、投资尽调等研究方法论，通过 Tool 接入外部数据源和内部知识系统，提升研究效率与结果可信度。

知识点：

- DeerFlow 主导 Agent：负责接收研究问题、拆解任务、编排执行流程
- Skill 机制：将研究方法论封装为可复用、可审查、可传承的技能包，同时针对Skill执行结果进行评估
- Tool 工具体系：接入搜索引擎、行业数据库、专利平台、Wiki、文档库、专家知识图谱等数据源
- 任务编排：完成信息搜集、交叉验证、结构化分析、报告生成
- 可追溯与审计：记录数据来源、引用依据、工具调用和执行 Trace
- 报告生成：支持 Markdown、HTML、可交互文档等多种输出形式

实现步骤：

1. 明确研究场景：确定平台支持的任务，例如竞品分析、行业调研、投资尽调、技术选型分析
2. 封装研究 Skill：将成熟研究流程沉淀为 Skill，定义输入、执行步骤、输出格式和质量标准
3. 接入数据源 Tool：接入外部数据源与内部知识系统，让 Agent 能自动获取研究所需资料
4. 构建研究 workflow：由主导 Agent 拆解任务，完成资料搜集、事实核验、结构化分析和报告生成
5. 建立证据链：记录引用来源、工具调用、分析过程和执行 Trace，保证结果可追溯、可审计
6. 输出研究报告：生成 Markdown、HTML 或可交互文档，适配技术团队、管理层和移动端使用场景
7. 优化效率与质量：通过 Skill 复用、模板化报告和人工反馈，让研究报告从数天产出缩短到数小时

第三部分：高阶案例二：新一代软件工厂

目标：

基于 DeerFlow 搭建新一代软件工厂，让 Agent 自动完成需求分析、任务拆解、代码开发、代码审查、测试验收和持续集成。通过 GitHub Channel、自定义 Agent 和 ACP 本地工具集成，构建端到端自动化开发流水线，提升软件交付效率与流程可控性。

知识点：

- DeerFlow Agent 编排：负责组织需求分析、任务分解、代码开发、测试和交付流程
- GitHub Channel：实现分支创建、代码提交、PR 发起、CI 检查等 DevOps 集成
- 自定义 Agent：定义产品经理 Agent、架构师 Agent、开发 Agent、QA Agent 等不同角色
- 多 Agent 协作：让不同角色 Agent 按职责分工完成复杂软件开发任务
- ACP 本地工具集成：连接本地构建、环境配置、命令执行等特殊任务
- 可追溯与审计：记录需求、代码变更、测试结果、PR 流程和 Agent 执行 Trace

步骤：

1. 明确软件工厂场景：确定平台支持的开发任务，例如需求分析、功能开发、Bug 修复、代码审查、测试生成、CI 检查
 2. 定义多角色 Agent：设计产品经理 Agent、架构师 Agent、开发 Agent、QA Agent 的职责、输入输出和协作边界
 3. 接入 GitHub Channel：让系统能够与代码仓库交互，自动创建分支、提交代码、发起 PR 并运行 CI 检查
 4. 构建自动化开发工作流：由主导 Agent 拆解任务，协调不同角色 Agent 完成方案设计、代码生成、代码审查和测试验收
 5. 集成 ACP 本地工具：通过 ACP 调用本地构建、环境配置、测试命令和项目脚本，实现跨平台任务编排。
 6. 建立流程追踪机制：记录任务拆解、代码变更、工具调用、测试结果、PR 状态和执行 Trace，保证过程可审计
 7. 优化交付效率与质量：通过角色分工、自动化流水线、人工确认和失败恢复机制，让软件开发从手工流程升级为智能工厂
-

第四部分：Agent 产品化与业务场景落地

Agent 场景判断

适合 Agent 的场景：

- 信息检索、文档分析、研究分析、代码理解、代码生成、客服辅助、销售运营、工单处理、流程自动化、企业知识问答

不适合 Agent 的场景：

- 极强确定性任务
- 高风险且不允许容错的任务
- 规则明确且脚本即可完成的任务
- 不允许人工审核的资金、医疗、法律决策
- 成本远高于人工的低价值任务

Agent、Workflow、RAG、Chatbot 与脚本的边界

- 何时使用 Chatbot
- 何时使用 RAG
- 何时使用 Workflow
- 何时使用 Agent

- 何时只需要自动化脚本
- 如何避免过度 Agent 化

业务流程拆解

- 从业务 SOP 拆 Agent Loop
- 从岗位任务拆 Tool
- 从知识资产拆 RAG
- 从经验方法论拆 Skill
- 从风险节点设计 Human-in-the-loop
- 从失败场景设计回滚
- 从业务目标定义 Eval

产品交付指标

- 任务完成率、人工接管率
- 平均处理时长
- 单任务成本、成本节省
- 自动化覆盖率
- 用户满意度、业务转化率
- 错误率、风险事件数

作品集与面试表达

- 如何将 Agent Engineering Platform 写入简历
- 如何描述 Tool Runtime
- 如何描述 Codebase Agent
- 如何描述 RAG 与 Context Engineering
- 如何描述 Benchmark 与 Eval
- 如何展示 Trace、Replay、灰度与成本治理
- AI Agent 开发工程师常见面试题
- 如何用项目数据证明工程能力
- 如何将技术项目表达为业务价值